REPORT ON
SPECIFICATION-DRIVEN DEVELOPMENT (SDD)
A structured approach to reliable AI-assisted software engineering

1. Introduction

Artificial intelligence is changing the way software teams plan and build applications. Coding assistants can suggest program logic, produce interface components, draft database queries, and help diagnose defects. These capabilities shorten some development activities, but speed alone is not a reliable measure of software quality.

A system can compile and still fail to solve the problem that motivated it. This often happens when the original request leaves important details unstated: who is allowed to perform an action, what should happen when data is missing, how errors are reported, or which limits must be enforced. An AI tool may fill these gaps with assumptions that seem reasonable but do not match the organisation's needs.

Specification-Driven Development (SDD) responds by making an agreed description of the intended system the starting point for engineering work. The team records the expected capabilities, operating rules, interfaces, limitations, and evidence required to demonstrate completion before asking developers or AI agents to implement the solution.

The specification is used throughout planning, coding, testing, review, and later enhancement. It gives product owners, engineers, testers, and AI tools a common reference. Prompts remain useful for requesting individual tasks, but they are written and evaluated in the context of the approved requirements.

The key principle is that implementation should follow the system's defined intent. The code is the mechanism used to deliver the required behaviour; it is not, by itself, the definition of what the product is supposed to do.

2. What is Specification-Driven Development (SDD)?

Specification-Driven Development is a method of building software in which a structured description of required behaviour guides design, implementation, and verification. Instead of beginning with an informal coding request, the team first clarifies what outcome is needed and how success will be recognised.

A specification may contain functional requirements, quality attributes, business rules, data definitions, interface contracts, assumptions, and acceptance

conditions. It should be detailed enough to remove important uncertainty while leaving room for engineers to choose suitable implementation techniques.
The specification is not simply a document prepared for approval and then ignored. It is maintained as an engineering reference. If a requirement changes, the specification is revised and the related design decisions, code, and tests are checked for impact.
Consider an online store that needs a wish-list feature. A short request such as 'allow customers to save products' does not explain whether guests can save items, whether duplicates are permitted, whether the list survives sign-out, or what happens when a product is discontinued. A specification can settle these rules before development begins.
For example, it could state that signed-in customers may save an available product once, remove saved products at any time, and see a clear notice if a saved item becomes unavailable. Those statements give implementation and testing teams a much clearer target.

2.1 Core principles of SDD

The approach is based on a few connected principles:
Clarity: describe required behaviour in direct language and define terms that could be interpreted in different ways.
Traceability: connect each important requirement to the design elements, implementation tasks, and tests that address it.
Consistency: ensure that components use compatible rules, data formats, and interface expectations.
Verifiability: express requirements so that reviewers can determine whether they have been met.
Controlled evolution: revise the specification when business needs change and record the effect on dependent work.
These principles do not require every technical choice to be decided at the beginning. They require important assumptions and constraints to be visible, reviewed, and updated when necessary.
3. Traditional Development vs Prompt-Driven Development vs Specification-Driven Development
Traditional software projects commonly progress from requirements analysis to design, implementation, testing, and release. Requirements

may be documented in a formal specification, user stories, process diagrams, or technical design records. The level of formality varies with the project.

Prompt-driven development uses an AI assistant as an active coding partner. A developer describes a task in ordinary language, receives generated code, and follows up with more prompts to refine the result. This can be productive for prototypes and isolated changes, but a long chain of conversations can make it difficult to preserve a single, consistent view of the product.

SDD combines the discipline of requirements engineering with AI-assisted implementation. The approved specification remains the reference across separate prompts, agents, and development stages.

Aspect	Traditional development	Prompt-driven development	Specification-driven development
Starting point	Requirements and design artefacts	A task described to an AI assistant	Reviewed and structured specification
Main reference	Project documents and engineering decisions	Conversation history and generated output	Current approved specification
AI contribution	Optional support tool	Often generates much of the code	Works within defined requirements and constraints
Consistency	Depends on document quality and team practices	May vary between separate prompts	Supported by shared rules and contracts
Testing	Usually planned as a distinct activity	May be requested after code is produced	Derived from requirements and acceptance conditions
Handling change	Update documents and affected code	Issue further prompts and repair inconsistencies	Revise the specification and trace affected work
Traceability	Depends on project process	Can be difficult to maintain	Explicit links between requirements and tests

The distinction is not simply whether a team writes more documentation. The important difference is whether the documented requirements actively

guide engineering choices and determine how the finished feature will be judged.

4. Why the Specification, Not the Prompt, Becomes the Source of Truth

A prompt and a specification serve different purposes. A prompt asks for an action in the current interaction. A specification records the agreed behaviour that the product must continue to satisfy across many interactions and releases.

A developer might ask an AI assistant to 'add a password reset screen'. That request does not necessarily explain how reset links expire, whether they can be reused, how many requests may be made, or how the system avoids revealing whether an email address has an account. The specification can define these rules once so that every implementation task follows the same expectations.

4.1 Prompts are temporary instructions

Prompts are usually scoped to an immediate activity. One prompt may request a form, another may ask for validation, and a later one may request a database change. When these instructions are treated as separate pieces of context, details can be missed or interpreted differently.

For example, one prompt might ask for a six-character password minimum while a later request introduces a stronger password policy. Without a maintained requirement, the frontend and backend may end up enforcing different rules. A persistent specification makes the current approved rule clear.

4.2 Specifications preserve engineering intent

Engineering intent describes the reason a feature exists and the outcome it must deliver. A prompt often focuses on the next coding step; the specification connects that step to the broader product objective.

A specification for password recovery could require that only the account owner can complete a reset, that the recovery token expires after a defined period, and that the user receives a neutral

confirmation message. The implementation can change over time, but these security and user-experience goals remain the standard.

4.3 Specifications reduce ambiguity

Words such as 'quick', 'reliable', and 'secure' are not precise enough to test on their own. A team should translate them into observable conditions wherever possible.

Instead of saying that a report should load quickly, a requirement could specify that it should display within two seconds for a defined dataset and test environment. Instead of saying that access must be secure, the specification can state which roles may view, create, or modify particular records.

Clear definitions reduce the number of decisions that an AI model or developer must guess.

4.4 Specifications support consistency across multiple AI agents

A project may assign different agents to user-interface development, API implementation, database work, test generation, and technical documentation. If each agent receives only a narrow prompt, their outputs may use different field names, status values, or assumptions.

A shared specification can define a common contract. For instance, it can require the API to return a field named 'reset_status' with one of a documented set of values. Both the frontend agent and backend agent then work against the same definition, reducing integration errors.

4.5 Specifications enable objective validation

A prompt can request a feature, but it does not automatically define how to decide whether the result is correct. A specification can provide measurable checks.

For a password reset feature, the checks might confirm that an expired token is rejected, a valid token can be used only once, and a successful reset allows the new password to authenticate. These conditions can become automated tests and review criteria.

4.6 Specifications improve change management

Requirements evolve as users, regulations, and business priorities change. If a company changes its account recovery policy, the team should update the specification and identify all affected screens, endpoints, data fields, and tests.

This is safer than relying on a new prompt alone because the team can see which existing behaviour is being replaced and which behaviour must remain unchanged. The revision also provides a record of the reason for the change.

4.7 Specifications strengthen accountability

A shared specification allows stakeholders to agree on expected behaviour before implementation. Product owners can confirm the business rules, engineers can plan the solution, and testers can verify the result against the same conditions.

The specification does not replace source code, code review, security assessment, or production monitoring. It is the reference for intended behaviour; evidence from implementation and testing is still needed to show that the system meets it.

5. Anatomy of a Good Specification

A useful specification gives enough information for a team to build and verify a feature without prescribing unnecessary implementation details. Six important parts are intent, scope, constraints, interfaces, acceptance criteria, and test cases.

5.1 Intent

Intent explains why a feature is required, which users it serves, and what improvement it should provide. It prevents the team from treating a feature as a collection of screens or database operations without understanding the user problem.

For a wish-list module, the intent may be to let customers keep track of products they are considering so they can return to them later without searching the catalogue again.

A strong intent statement identifies the user, the problem, and the desired outcome. It should communicate purpose without locking the team into a particular framework or database design.

5.2 Scope

Scope sets the limits of the work. It lists what the current release will include and what it will deliberately leave out. This is important because AI tools may suggest extra capabilities that sound useful but increase complexity and testing effort.

A first version of a wish-list feature might include adding a product, removing a product, viewing saved products, and retaining the list for signed-in customers. Social sharing, price-drop alerts, product recommendations, and multiple named lists could be deferred.

The scope should also record assumptions and dependencies, such as the availability of a product catalogue API or an existing user authentication service.

5.3 Constraints

Constraints are conditions that the solution must respect. They may come from the organisation's technology stack, security policy, performance expectations, deployment environment, or compatibility requirements.

Technical constraints: approved programming languages, frameworks, databases, and hosting platforms.

Performance constraints: response-time targets, concurrent-user expectations, and resource limits.

Security constraints: authentication, permissions, encryption, and safe data handling.

Operational constraints: logging, backup, monitoring, deployment, and recovery requirements.

Compatibility constraints: supported browsers, devices, operating systems, and API versions.

A constraint should be testable when possible. 'The service should be reliable' is broad; an agreed availability target measured over a stated

period is more useful. Constraints should distinguish mandatory decisions from preferences.

5.4 Interfaces

Interfaces define how users and software components exchange information. They cover user actions as well as communication between services, databases, and external systems.

An interface description may specify endpoint paths, HTTP methods, input and output fields, data types, permitted values, authentication rules, and error responses. For a wish-list API, the contract should explain how a product is identified, what response is returned after saving it, and what happens if the product no longer exists.

Consistent interface definitions allow separate teams or agents to work in parallel. They also make it easier to identify incompatibilities before components are integrated.

5.5 Acceptance criteria

Acceptance criteria state the conditions that must be true before a feature can be approved. They turn a broad requirement into behaviour that can be observed and checked.

One practical way to write them is Given–When–Then:

Given: the starting condition or context.

When: the user action or event.

Then: the expected outcome.

Example: Given that a signed-in customer is viewing an active product, when the customer selects 'Save to Wish List', then the product is added and appears in the customer's saved-products page.

Criteria should cover relevant success paths as well as restrictions, such as preventing duplicate entries or handling unavailable products.

5.6 Test cases

Test cases describe the steps, inputs, and expected results used to check that requirements have been implemented correctly.

Acceptance criteria describe the target behaviour; test cases provide a repeatable way to verify it.

A wish-list test plan could check a valid save operation, saving the same product twice, removing an item, opening the list after signing out and back in, and attempting to save a product that has been deleted.
Tests should include normal, invalid, boundary, and failure conditions. AI-generated tests must be reviewed because they can repeat an incorrect assumption from the generated code. Each test should map to one or more requirements or acceptance criteria.
6. Practical Example: Specification for an E-Commerce Shopping Cart
The following example demonstrates how the six specification elements can be applied to a shopping cart. It uses a fresh set of rules and scenarios to illustrate the same subject area.
6.1 Feature overview
Field Description
Feature name Shopping Cart and Order Preparation
Specification ID SDD-ECOM- CART-014
Version 1.1
Objective Allow a customer to review selected items, change quantities, remove unwanted products, and prepare a valid order.

6.2 Intent
The cart should give customers a dependable summary of the items they intend to buy. It must show current product details, make quantity changes understandable, and display a total calculated using trusted product prices. The customer should be able to identify problems before moving to checkout.
6.3 Scope
Included in this version:
Add an active catalogue item to the cart.
Increase or decrease the quantity of an item.
Remove a selected item from the cart.
Display item name, unit price, quantity, and line total.

Recalculate the cart summary after every successful change.
Prevent checkout when the cart is empty or contains unavailable items.
Not included in this version: loyalty-point redemption, gift wrapping, split shipments, coupon stacking, and live payment processing. These capabilities can be specified separately in a later release.

6.4 Constraints

The browser interface will be implemented with React and will communicate with a REST service.
The backend service will use FastAPI, with product and inventory information stored in PostgreSQL.
A quantity must be a whole number greater than zero and must not exceed the available stock.
The server must calculate monetary values using integer minor units (paise) rather than relying on browser-submitted totals.
A cart update must not succeed if the product has been discontinued or the requested quantity is no longer available.
The API must require an authenticated customer for persistent cart operations.

6.5 Interfaces

Method Endpoint Purpose
GET /api/cart Return the current customer's cart items and summary.
POST /api/cart/items Add a catalogue product with a requested quantity.
PATCH /api/cart/items/{item_id} Change the quantity of an existing cart line.
DELETE /api/cart/items/{item_id} Remove a cart line.
POST /api/cart/validate Check that all cart items remain eligible for checkout.

Example request to add an item:

```
{
  "product_id": 208,
  "quantity": 2
}
```

Example success response:

```
{
  "status": "added",
  "item_id": 734,
  "product_id": 208,
  "quantity": 2,
  "unit_price_paise": 74950,
  "line_total_paise": 149900,
  "currency": "INR"
}
```

In this example, 74,950 paise represents ₹749.50 per item, and the line total for two items is ₹1,499.00. The server must obtain the unit price from the product record rather than accepting a price supplied by the client.

The contract should define consistent error behaviour. For example, malformed quantities may return HTTP 422, an unknown product may return HTTP 404, and a stock conflict may return HTTP 409. These are proposed choices for this example and should be confirmed by the project team.

6.6 Acceptance criteria

ID	Acceptance criterion
AC-01	An authenticated customer can add an active product to the cart.
AC-02	The system rejects a quantity that is zero, negative, or not a whole number.
AC-03	The system prevents a cart quantity from exceeding current available stock.

AC-04 Changing or removing a cart line refreshes the displayed subtotal.
AC-05 The server calculates line totals and the cart total from stored product prices.
AC-06 Checkout validation fails when the cart is empty or contains an unavailable item.
AC-07 A customer's persistent cart is not visible to another customer.

6.7 Test cases

Test ID	Scenario	Expected result	Maps to
TC-01	Add product 208 with quantity 2	The item is returned in the cart with quantity 2.	AC-01
TC-02	Submit quantity 0	The API rejects the request with a validation response.	AC-02
TC-03	Request 12 units when only 8 are available	The update is rejected and the existing cart remains unchanged.	AC-03
TC-04	Change quantity from 2 to 3	The line total and cart subtotal are recalculated.	AC-04, AC-05
TC-05	Remove the only item	The cart is empty and its subtotal is zero.	AC-04
TC-06	Modify the browser request to send a lower price	The server ignores the client price and uses the catalogue price.	AC-05
TC-07	Validate an empty cart for checkout	Checkout validation returns an unsuccessful result.	AC-06
TC-08	Attempt to retrieve another customer's cart	Access is denied and no other customer's cart data is returned.	AC-07

This example links a business objective to explicit system rules, interface definitions, and verification steps. An AI assistant can be asked to implement a particular API or interface component, but the

generated result should be checked against these criteria rather than judged only by whether the screen appears to work.

7. Specification-Driven Development Workflow

SDD can be organised as a repeatable sequence. The activities may overlap in an iterative project, but each stage should leave behind clear information for the next one.

1. Define intent and requirements

Meet with stakeholders to identify the problem, target users, expected outcomes, and essential business rules.

2. Create and review the specification

Record scope, constraints, interfaces, assumptions, acceptance criteria, and unresolved questions. Review the content with the people who own the requirements.

3. Plan architecture and tasks

Break the approved requirements into components, data flows, dependencies, implementation tasks, and review responsibilities.

4. Implement with AI assistance

Provide the relevant specification sections to developers or AI agents. Generate code in manageable units and keep changes within the agreed scope.

5. Validate against the specification

Run unit, integration, and end-to-end tests; inspect security and error handling; and compare observed behaviour with the acceptance criteria.

6. Review and refine

Resolve gaps, update requirements when approved changes occur, and repeat the affected planning, implementation, and verification activities.

A practical workflow is iterative rather than strictly one-way. Testing may reveal a missing requirement, or implementation planning may expose a conflict between two constraints. The team should return to

the specification, clarify the issue, and record the agreed resolution before proceeding.

The important control is that code changes and requirement changes remain connected. A change should not silently alter the product's intended behaviour.

8. Benefits of Specification-Driven Development in AI-Assisted Engineering

8.1 Improved requirement clarity

Writing down expected behaviour encourages stakeholders to discuss edge cases and resolve unclear terms before those uncertainties become code. This improves the quality of conversations between business and technical teams.

8.2 Reduced implementation rework

When developers understand the expected result early, they are less likely to build a feature around an incorrect assumption. This can reduce repeated code changes, especially where several components depend on the same rule.

8.3 More reliable AI-generated code

AI tools receive a more complete context when they are given defined interfaces, constraints, and acceptance conditions. The specification does not guarantee that generated code is correct, but it gives reviewers a concrete basis for finding omissions and inconsistencies.

8.4 Better test coverage

Requirements can be translated into test scenarios before implementation is complete. Teams are therefore more likely to consider invalid input, permission boundaries, unusual quantities, and service failures in addition to the normal user journey.

8.5 Easier collaboration

A shared reference reduces the risk that frontend, backend, QA, and product teams are working from different interpretations. It is also useful when several AI agents contribute to separate parts of the same application.

8.6 Improved maintainability and traceability

A maintained specification helps future engineers understand the reason for a rule and identify which tests protect it. When a requirement changes, traceability makes it easier to locate dependent components and assess the impact.

9. Challenges and Limitations of SDD

SDD has costs as well as benefits. Preparing a useful specification requires time for discussion, analysis, and review. For a small throwaway script or a short experiment, a detailed specification may create more overhead than value. Teams should choose a level of formality that matches the risk and complexity of the work.

The quality of the output is limited by the quality of the specification. A document can be well formatted and still contain incorrect assumptions, missing user needs, or conflicting requirements. Stakeholder review and domain knowledge remain essential.

Another risk is specification drift. This occurs when code is changed but the specification is not updated, leaving the documented rules out of sync with the real application. Version control and change review help reduce this problem.

Change management can also become demanding. A modification to one business rule may affect APIs, database fields, user interfaces, reports, and automated tests. Teams need a process for assessing these dependencies before approving changes.

Finally, AI-generated code must not be trusted solely because it follows a specification. The specification may omit a security threat or an unusual operating condition, and the generated implementation may still contain defects. Human review, independent testing, security checks, and production monitoring remain necessary.

10. Best Practices for Writing Effective Specifications

Use specific language. Replace vague descriptions such as 'easy to use' or 'very fast' with observable behaviour or measurable targets.

Give important requirements unique identifiers so they can be referenced in tasks, code reviews, and tests.
Separate required behaviour from optional ideas and future enhancements.
Record assumptions, dependencies, exclusions, and unanswered questions rather than allowing them to remain implicit.
Define interfaces as contracts, including field names, data types, valid values, authentication, and error behaviour.
Write acceptance criteria from the user's or system's observable perspective.
Include positive, negative, boundary, permission, and failure test scenarios.
Keep the specification under version control and review changes in the same disciplined way as code changes.
Provide AI agents with the relevant approved context and instruct them not to invent requirements when information is missing.
Review generated code and tests independently; passing tests are useful evidence only when the tests represent the intended requirements.
The specification should be treated as a living engineering artefact. It needs to evolve with the product while remaining aligned with architecture, implementation, and verification evidence.

11. Conclusion

Specification-Driven Development provides a structured way to manage software work in an environment where AI can generate code quickly. It addresses a central challenge of AI-assisted engineering: producing an implementation that matches the actual needs of users and stakeholders, not merely one that appears plausible from a short instruction.
By defining intent, scope, constraints, interfaces, acceptance criteria, and test cases, a team establishes a common understanding of the system before implementation begins. That reference can guide

developers and AI agents through planning, coding, integration, testing, and future changes.

Prompts continue to play an important role because they direct AI tools toward specific tasks. However, prompts should operate within the boundaries of the approved specification. The specification preserves the product's intended behaviour across multiple conversations, contributors, and releases.

SDD does not remove the need for engineering judgement or human oversight. Its value is that it makes expectations explicit, gives teams a basis for objective verification, and helps connect every implementation decision to a stated requirement. The result is a more disciplined path from an initial idea to software that can be tested and shown to meet its intended purpose.